

The Price of Predictability: What a Machine-Checked Contract Buys — and Costs — in Agentic Code Repair

Withheld for review

Study repository and full data available; see Data Availability

Abstract. Formal Contract-Driven Development (FCDD) organises an AI coding agent’s work around a machine-checked contract. Its purpose is *predictability*: a repair pinned to a kernel-checked specification should land in a narrow band of behaviour and cost, and should not drift into hallucinated or unfounded changes. This study asks two questions in that order: does the pinning work, and what does it cost? We ran a controlled comparison on a 7,343-line Z80 assembly chess engine into which an independent agent seeded seven single-byte faults, each with a sealed acceptance test written before any repair was attempted; two arms fixed every fault. *On cost the answer is unambiguous*: FCDD was more expensive on 7 of 7 matched pairs (exact two-sided sign test $p = 0.016$, the study’s only statistically supportable result), median ratio $4.74\times$ on raw tokens (the pre-registered primary metric), and its upfront contract never amortised. *On predictability the data is suggestive and insufficient*: the across-defect dispersion contrast rests entirely on one cell — excluding it the two arms’ spreads are identical at $2.31\times$ — and replicating that cell showed the ordinary arm forking between a minimal repair (\$11.78) and a below-gate-floor redesign (\$44.27, \$41.35) while FCDD produced the minimal repair in all three runs, at a replication depth where no dispersion statistic can reach significance (Fisher $p = 0.40$; attainable floor 0.10). The quality comparison’s paired CI, $[-0.44, +0.78]$, spans zero. The honest summary is asymmetric: the price is demonstrated; the product is not yet. We also report three measurement findings we regard as more transferable than either ratio: an instruction to review “until no findings remain” has no fixed point; measured cost covaries with review-loop length at least as strongly as with method; and processes killed by infrastructure emit no cost record, under-counting one cell by roughly $20\times$.

Keywords: Formal methods · Empirical software engineering · Predictability · LLM agents · Cost measurement · Code review

1 Introduction

Agentic AI code repair has a predictability problem before it has a cost problem. An unconstrained agent asked to fix a fault may fix it minimally, or redesign the subsystem around it, or “fix” code that was never broken; runs of the same

task have been measured to differ by up to 30× in token consumption [5], and agent-reported success diverges widely from human-confirmed success [28]. For a maintainer, the expected cost of a fix matters less than the variance of what they will receive.

Formal Contract-Driven Development (FCDD) is a response to that problem, not to the cost problem. It surrounds the agent with a machine-checked contract — a kernel-checked specification the fix must name, satisfy and extend — on the theory that a repair pinned to named clauses cannot silently become a redesign, and a change with no clause behind it is visible as such. That is the design intent. Whether the pinning *works* is an empirical question, and it is the first of the two questions this study asks:

- **Goal 1 — predictability.** Does the contract make the agent’s repairs land in a narrower band — of solution strategy, of quality, of cost — and does it reduce unfounded changes?
- **Goal 2 — cost.** What does the pinning cost, per defect, against a competent ordinary alternative doing the identical repair to the identical gate?

The intended thesis, stated before the data so the reader can watch it take damage: *predictability is a trade-off purchased with elevated cost* — the ordinary approach is cheap when the problem is easy but unpredictable and occasionally very expensive, whereas the formal approach costs more up front and lands in a narrow band. Section 6 tests each half separately. The cost half survives intact. The predictability half survives only as a directional observation on one replicated defect, and we say so plainly: this experiment was powered to price the method, not to prove its purpose.

A second contribution is methodological. Three times a striking result proved to be an artefact of our apparatus rather than a property of the methods; a fourth and fifth were caught only by adversarial review of the finished draft, and this revision added a sixth (a published regression that did not reproduce; Sect. 5.3). Each correction moved a headline, and two reversed its direction. Section 5 is therefore placed before the results.

2 The Method Under Test

2.1 The two arms

Ordinary development-plus-review (Arm A). Reproduce the symptom, fix the fault, add a test, obtain a review from a fresh reviewer, address its findings; stop when the pre-registered gate passes. No formal-methods vocabulary appears in the instructions.

FCDD (Arm B). The same bug report and gate, but the session works around a pre-built machine-checked contract, following the lifecycle in Fig. 1.

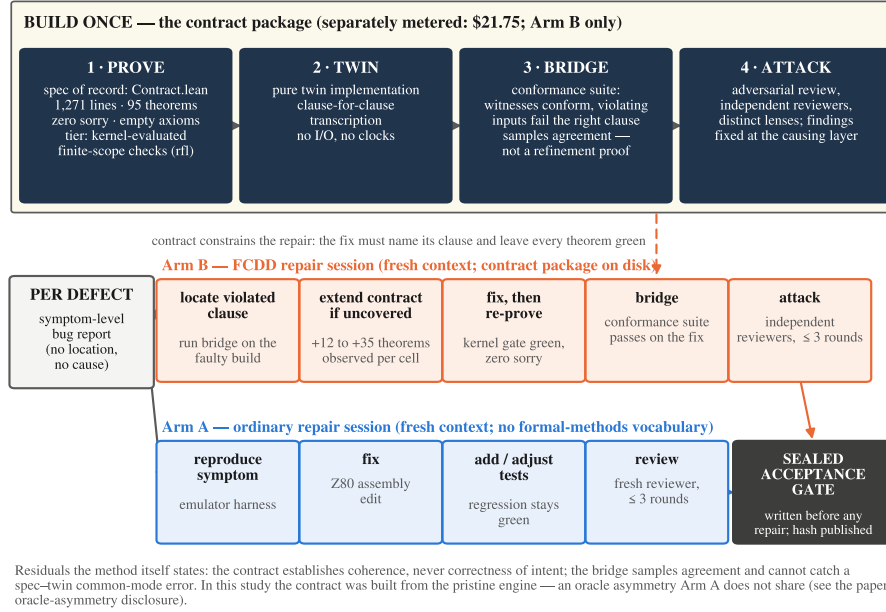


Fig. 1. FCDD as executed in this study. Top: the build-once contract package (four beats, each leaving a distinct artefact; \$21.75, metered separately as Arm B’s upfront investment). Middle: the per-defect FCDD repair pipeline the contract constrains. Bottom: the ordinary arm’s pipeline. Both lanes end at the same sealed acceptance gate, written before any repair began.

2.2 The four beats and their artefacts

FCDD builds the contract once, in four beats (Fig. 1, top). (1) **PROVE**: the safety properties are formalised as the *spec of record* in a kernel-checked prover — here Lean 4 core, producing `Contract.lean`: 1,271 lines, 95 theorems, zero **sorry**, empty axiom profile. (2) **TWIN**: a *pure twin implementation* — side-effect-free, no I/O — transcribed clause-for-clause from the spec. (3) **BRIDGE**: a *conformance suite* connecting twin to spec and to the real engine through the emulator: witnesses must conform, inputs that violate a theorem must fail the *right* clause, and single-field mutations must trip exactly their clause. The bridge *samples* agreement; it is not a refinement proof, and the method says so. (4) **ATTACK**: multi-agent adversarial review with distinct lenses, findings verified by execution and fixed at the layer that caused them.

What tier of verification this actually is. All 95 theorems are discharged by `rfl` — kernel evaluation of a closed decidable Boolean on a concrete witness. There is no universally-quantified theorem, no induction and no invariant proof in the contract. The tier is *kernel-evaluated finite-scope checking*: the Lean kernel

used as a trustworthy test runner. The consequence favours the paper’s own cost headline: we priced the cheapest formal tier, so our cost ratio is a *lower bound* on what deductive verification would have cost. The artefact belongs with lightweight formal methods rather than with seL4-style proof.

2.3 How the contract constrains the repair agent

The per-defect FCDD session (Fig. 1, middle) receives the same symptom-level bug report as Arm A, plus the contract package on disk. Its instructions require it to: run the bridge against the faulty build and *locate the violated clause*; if no clause is violated, *extend the contract* until one is (every one of the seven cells did extend it, by +12 to +35 theorems; Sect. 4.2); fix at the offending layer; *re-prove* (the kernel gate must return green over all theorems, old and new); re-run the *bridge*; and submit to an *attack round* of independent reviewers, capped at three rounds like Arm A’s review.

This is the mechanism by which predictability *could* arise, stated here as design rather than result: a redesign must leave every named theorem green, which prices deviation from specified behaviour; a change with no clause behind it is structurally visible; and the seeded fault itself becomes a permanent negative test. Whether any of that shows up in the data is the question of Sects. 6.2 and 6.3.

2.4 What the method itself disclaims

FCDD’s own documentation states two residuals that bound everything this study can conclude: the contract establishes *coherence, never correctness of intent* — garbage clauses, kernel-checked, are still garbage — and the bridge cannot catch a *common-mode error* in which specification and twin share one author’s misconception. Both residuals became measurable events in this study (Sect. 4.2’s vacuous antisymmetry clause; Sect. 6.4’s gate common-mode), so they are not hypothetical.

3 Related Work

Empirical cost of formal methods. The field’s own guidance notes that empirical studies of formal methods are rare [6]. What exists is mostly retrospective single-project accounting with no control arm: seL4 needed 2.2 person-years to build against about 20 to verify [33,17]; ShardStore’s authors compare their overhead to figures *drawn from other papers* [9]; the AWS TLA+ reports rest on counterfactual testimony [21]; CompCert reports cost without comparison [19]. The largest survey concedes the gap outright: in a majority of projects surveyed there is no data available on relative time and costs [36]. The classic industrial comparison is observational and unpriced [25].

The closest precedent. Sobel and Clarkson compared formal and informal development across student teams [31]. Berry and Tichy characterise it as a quasi experiment — “assignment was voluntary and not random” — and called for a complete redesign [8]; the authors’ rebuttal argues theirs was field rather than laboratory research [32]. Theirs is the closest prior comparison; it was not randomised and it priced neither arm. Our sealed acceptance tests and order-counterbalanced grading respond directly to that critique. The closest work in unit of analysis reports 87 minutes to develop each of 73 unit proofs against recreated defects [2] — per-artifact cost, but against a detection-rate baseline, in human minutes rather than tokens.

Agentic repair, its cost, and its variance. LLM agents repairing real defects is established ground [13,39,38], as is the finding that simpler structured pipelines can beat autonomous agents on quality and dollars — Agentless reports \$0.70 per issue [37]. Our contribution is not “structure helps” but *which* structure, at what price. Cost-aware agent evaluation is prior art [15], and same-task token consumption can differ by up to $30\times$ [5] — a noise floor any cost delta must clear, and the documented form of the unpredictability Goal 1 targets. Benchmark contamination motivated seeding our own faults [1]; agent repair evaluated inside Google shows a large gap between machine-reported and human-reported defects [28]. The nearest neighbour varies one prompt line controlling specification grounding for +38 percentage points of correct code [11], but reports no cost accounting.

Review effectiveness. Review surfaces far more maintainability issues than defects: 75% of review findings do not affect visible functionality [20], replicated in open source with 7–35% of suggestions producing no change [7], anticipated by the expectation–outcome gap [4]. Porter et al. found team size and session count did not significantly move defect detection rate [26]; the per-review signal-to-noise figures we match are Rigby and Bird’s derivation from that data, and they report that more reviewers yield “only a minimal increase in the number of comments” [27]. Our 85 findings without a defect in the fix is therefore a replication. What appears unclaimed is the *stopping rule*: over 80% of human changes involve at most one review iteration [29], so twelve rounds under an unbounded instruction is an order-of-magnitude departure.

Seeded faults — a threat we inherit. Mutants couple to 73% of real faults and 17% of real faults couple to no mutant [14]; correlation with real-fault detection weakens once suite size is controlled [24]; uncontrolled mutation-based designs incur Type I errors $\sim 62\%$ of the time [23]; hand-seeded faults tend to be harder to detect than real ones [3]. Our defence is that single-statement bugs are an abundant measured class [16].

Position bias in LLM grading. Position bias is established [40,34], strongest when the quality gap is small [30] — exactly our regime. Our first grading

round showed 4 of 4 pairs favouring the second-presented submission; on re-examination that pattern is fully explained by a genuine arm effect under the pre-registered parity alternation, so the honest statement is that our first design *could not have detected* position bias, not that we observed it. We retain the counterbalanced design because it removes the confound we could not otherwise rule out. The instrument’s SE-specific validation is partial, and the unvalidated part cuts against one of our own outputs: the source validates LLM-as-judge for output-based *direct scoring* on code translation and generation, but concludes that current methods “fail to deliver satisfactory and consistent comparison performance on SE tasks” under *pairwise* comparison [35]. Our per-axis rubric falls on the validated side of that line; the holistic pairwise verdict of Sect. 6.3 falls on the side the source argues against — a source-backed reason, beyond its non-registration, to weight the pre-registered axes over that verdict. For method comparison we follow standard guidance [18] and report effort as a separate dependent variable rather than equalising it [12].

What remains ours. Searching for prior work pricing a formal-methods workflow in LLM token cost against a conventional one returned nothing we could verify; we state that as a searched-and-not-found result.

4 Experimental Method

4.1 System, faults, execution

The subject is a chess engine of 7,343 lines of hand-written Z80 assembly: alpha-beta search, transposition tables, quiescence, Zobrist hashing, built-in perft. An independent agent seeded seven single-fault variants, each differing from pristine by *exactly one byte* in the assembled binary, wrote symptom-level reports with no location hints, and sealed an answer key whose hash was published before any repair began. Each arm×defect cell ran as an isolated headless session on the same model at the same effort, with no network access.

Which version of the method was measured. Arm B loads the method at run time—its prompt directs the agent to invoke the `formal-contract-dev` skill before working—so the arm is bound to whatever that document said during the runs. That document is mutable and has since changed. The version measured here is archived in the study repository at `skills/formal-contract-dev/SKILL.md` (commit `4ca5bb3`, 2026-08-04; 436 lines; sha256 `68e384df562bad87...`), and every number in this paper refers to it. The installed copy has since gained one additional item (16.5, a solver/reviewer serialisation turnstile, added 2026-08-07 in response to an unrelated incident); the change is purely additive, so re-running Arm B today would follow strictly more process than we measured, which can only raise its cost. FCDD’s contract was built once as a separately measured phase at \$21.75.

4.2 An asymmetry we did not design and must disclose

The contract was built by reading the *pristine* engine, so Arm B held a derived oracle for correct behaviour that Arm A never received; Arm A’s only prior artefact was the symptom report. The asymmetry recurs in the interpretation of every Goal-1 result below, because a “consistent” repair guided by a pristine-derived oracle may be consistency purchased by the oracle, not by the method’s structure.

An earlier draft claimed the contract “covers 7/7 faults by construction”; that is false and withdrawn. Diffing each cell’s specification against the 95-theorem baseline (re-counted mechanically for this revision; the counts match) shows every Arm B cell had to *extend* the contract: bug07 +12, bug02 +16, bug05 +18, bug01 +29, bug06 +29, bug04 +33, bug03 +35 theorems. The rule-layer clauses C1–C14 covered the rule-layer faults; the search and evaluation layer was not covered and had to be specified from scratch.

One case deserves its own report. The baseline contained an evaluation-antisymmetry property, $evalWhite(mirror\ p) = -evalWhite\ p$. A global sign inversion satisfies that identity exactly — the pristine-derived contract was *vacuous* against bug04’s entire fault class, and the arm had to discover this and add absolute-orientation anchors. That is a too-weak-specification result obtained under controlled conditions, and it is evidence against the very oracle advantage we are disclosing.

Two weaker mechanism statements survive re-derivation (an earlier draft’s grouping — “four cells needing new properties, median \$49.16 vs \$24.19” — could not be reproduced from the deposited artefacts and is withdrawn). First, the number of theorems a cell added does *not* predict its cost (Spearman $\rho = 0.03$; bug03 added the most theorems and was nearly the cheapest). Second, the three faults seeded in the search/evaluation layer, which the baseline did not cover, have a median cost of \$49.63 against \$27.62 for the four rule-layer faults — a mechanism consistent with coverage-driven cost, not a tested effect ($n = 7$, and one rule-layer cell sits in the wrong group).

The fair design would have given Arm A a cost-matched upfront artefact — roughly \$21.75 of characterisation tests built once from the pristine engine — and we did not. This advantages FCDD on localisation, and it would not transfer to faults of omission, which lie outside any contract by definition.

4.3 Measurement

Cost is reported three ways, because they disagree and the disagreement matters: the platform’s figure; the same token counts priced with our pre-registered rate table; and raw token totals. Token totals come from each session’s `modelUsage`, which includes reviewer subagents — an earlier ledger recorded the main-loop `usage` field only, under-counting 22 rows by $1.1 \times -82 \times$ until repaired. Reviewer invocations are counted mechanically from transcripts, because (Sect. 5.3) they covary with cost as strongly as method does.

4.4 Pre-registration and its breaches

The protocol was frozen before any run; changes are dated amendments (A1–A10). Two breaches: *rule 5 (replication) was honoured late* — bug01 triggers it (arm gap 10.8%, under the 30% trigger) and the previous draft went to review with the rule undischarged; the replication has since been completed and Sect. 6.2 reports it. And *the metric was substituted without amendment*: D3 froze a rate table and legislated raw-only for unpriced models; the model used is unpriced there, yet we reported platform dollars. Reverse-engineering the platform’s rates from 41 runs matches D3 within $\sim 6\%$, so the substitution is materially harmless — but it was undocumented.

5 Instrument Findings

5.1 Reviews that never closed

Subagents defaulted to background execution and a headless session ends when its main loop ends, so sessions terminated with reviews pending; five cells self-reported waiting for reviewers. The stage was under-executed and under-counted, symmetrically across arms.

5.2 Unbounded review has no fixed point

The corrected instruction read *iterate until the reviewer has no findings left*. That rule has no fixed point — a competent reviewer examining any test suite can always name one more uncovered case. In one cell Arm A fixed the seeded fault correctly on its first pass (one byte, `cp WN` $\rightarrow$ `cp WP`), then ran *12 review rounds producing 85 findings, none of which was a defect in the fix*. Findings per round: 7, 7, 5, 11, 8, 9, 5, 5, 5, 3, 8, 12 — OLS slope $+0.025/\text{round}$, $R^2 = 0.001$. No convergence. An earlier draft said we halted this run; the record shows an `api_error` 429 (session quota) and a second attempt of \$67.94 our accounting missed. Summed per our own rule the cell cost at least \$128.62 across 7.7 hours; the bounded re-run cost \$7.32 — a reduction of at least $17.6\times$.

5.3 Cost covaries with review effort at least as strongly as with method

An early draft claimed cost “tracked reviewer count more tightly than which arm was running”, from five cells in which arm and count were collinear — not identifiable. *The regression published in the previous draft is itself corrected here*: it printed $\text{cost} = 4.13 + 21.35 \cdot \text{ArmB} + 7.13 \cdot \text{rev}$, $R^2 = 0.826$; re-deriving from the deposited table (13 final-instrument cells with a deposited platform cost) gives different coefficients, and no subset of the deposited data reproduces those numbers. The corrected fit (Fig. 2):

$$\text{cost} = 1.56 + 16.85 \cdot \text{ArmB} + 8.76 \cdot \text{reviewers}, \quad R^2 = 0.873.$$

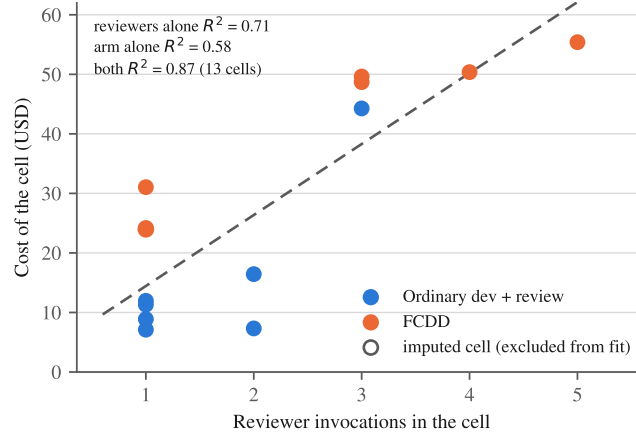


Fig. 2. Cell cost against reviewer invocations, 13 final-instrument cells; the imputed bug03/B cell is hollow and excluded from the fit. The bounded rule caps review *rounds*, not invocations; FCDD’s attack beat is multi-reviewer per round, so the rule granted Arm B up to five invocations and Arm A at most three.

Arm alone $R^2 = 0.580$ (gap \$27.90); reviewers alone $R^2 = 0.711$. Controlling for reviewer count attenuates the arm gap from \$27.90 to \$16.85: roughly 40% of the measured method difference is attributable to Arm B invoking $1.8\times$ as many reviewers (mean 2.83 vs 1.57). The correction *strengthens* the section’s claim — reviewers alone now explain more variance than arm alone — but that is luck, not vindication: a published regression failed to reproduce and we cannot reconstruct where its coefficients came from.

A further correction, retained: bounding rounds did not compress Arm A’s range to “\$7–\$16” as once claimed — its costs are \$7.11–\$44.27, and the outlier is the cell with three reviewers against \$8.66 with one. Bounding rounds did not bound reviewer invocations, and cost followed the latter.

5.4 Killed processes emit no cost record

One cell reported \$1.06; its first attempt had been killed without emitting a result record. Recovering 34.67M tokens from the transcripts and pricing them at our pre-registered rates gives \$23.98 (band \$23–\$33) — an under-count of $\sim 20\times$ (an earlier draft said \$45.76 and $43\times$, having borrowed a blended rate from a cell whose own denominator was truncated). A second failure mode sits in our own ledger: one record carries \$49.19 with all four token counters at zero. Exit reports can be internally self-contradictory, not merely absent.

Table 1. Cost per defect, final instrument. † imputed from recovered tokens at pre-registered rates. “rev” = reviewer invocations.

Defect	Arm A	Arm B	Ratio	A tok	B tok	A rev	B rev
bug01 quiescence	\$44.27	\$49.63	1.12×	49.3M	58.3M	3	3
bug02 fifty-move	\$7.32	\$50.37	6.88×	5.5M	58.3M	2	4
bug03 promotion	\$11.96	\$23.98†	2.01×	13.7M	34.7M	1	1
bug04 eval sign	\$11.30	\$55.40	4.90×	12.2M	67.4M	1	5
bug05 castling	\$8.88	\$31.05	3.50×	8.9M	40.5M	1	1
bug06 TT bound	\$16.44	\$48.69	2.96×	16.5M	57.8M	2	3
bug07 mate detect	\$7.11	\$24.19	3.40×	5.7M	31.9M	1	1
Median (tokens / platform / rates)				4.74× / 3.40× / 3.28×			

5.5 Blinding defects in grading

Our first grading round was voided: submissions were diffed against pristine, but a correct minimal repair *restores* pristine, so the repair was invisible; FCDD’s verification artefacts were omitted, so FCDD scored 1/5 for testing on the grounds that it shipped no tests; fixing that leaked paths naming the arm. *The regraded round is not blinded, and we quantify that rather than hedge it*: counting treatment-signature terms (theorem, specification, clause, solver, witness, mutant) in one packet’s two submissions, at comparable length: 0 occurrences for the ordinary arm, 37 for FCDD. That is not a residue; it is a label. “Blinded grading” is withdrawn as a competing-interest mitigation. The accurate description of Sect. 6.3 is: *unblinded graders preferred the submission that visibly contains the treatment*. Compounding this, on three of seven defects the source fixes were byte-identical, so the verdict turned entirely on the visible artefact, and the test-quality axis mechanically rewards shipping more verification material — which is the treatment’s definition.

6 Results

6.1 Cost (Goal 2)

All seven pairs completed under the final instrument (Fig. 3, Table 1). *FCDD was dearer on 7 of 7 pairs; exact two-sided sign test* $p = 0.016$ — the only statistically supportable result in this study.

Headline metric. Our pre-registration (D3) legislated for exactly this model: “Opus 5 unpriced $\rightarrow$ raw-only if it ever appears.” Every measured row is that model. The primary figure is therefore the raw-token median, 4.74×; platform dollars (3.40×) and our rate table (3.28×) are robustness checks. Earlier drafts inverted this — headlining the metric D3 forbade, in a paper whose Sect. 5 proves platform self-reports here can be absent, truncated, zero-token or self-contradictory. *Interval*: an earlier draft reported a “98% CI [2.01, 4.90]”. That

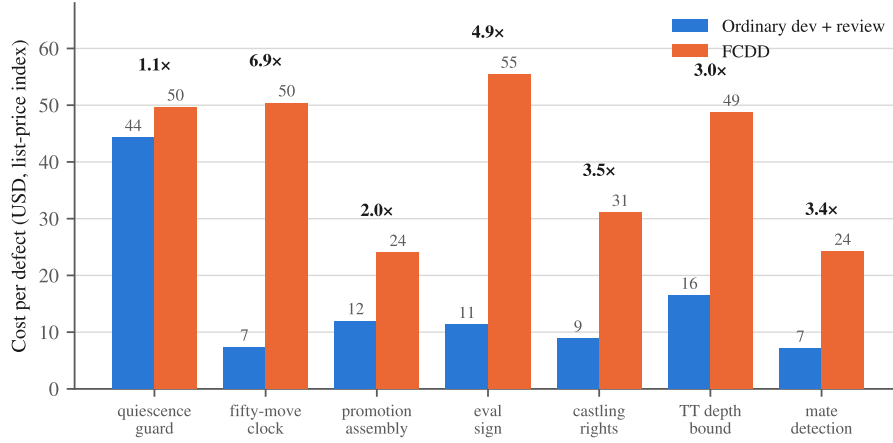


Fig. 3. Cost per defect. FCDD is dearer on every pair; the ratio runs from 1.1 $\times$ on the defect where Arm A happened to converge on its expensive strategy (Sect. 6.2) to 6.9 $\times$ on the simplest.

was an off-by-one in our own `median_ci` — the true coverage of $[2.01, 4.90]$ is 87.5%; at $n = 7$ the tightest nonparametric interval reaching 95% is the full observed range, $[1.12, 6.88]$ (98.4%). The bug was found by adversarial review of the script, not by our own checks.

The 7/7 result depends on one author-computed imputation. Taken exactly as deposited, bug03/B is \$1.06 against Arm A’s \$11.96 — Arm B *cheaper* — giving 6/7 and $p = 0.125$. The 7/7 exists because we recovered that cell’s transcript tokens and priced them; no other cell received transcript-level recovery, and no imputation rule was pre-registered. Sensitivity: deposited values $\rightarrow$ 6/7, $p = 0.125$; imputed $\rightarrow$ 7/7, $p = 0.016$; completing-attempt proxy (defined for 4 of 7 cells) $\rightarrow$ 4/4, $p = 0.125$. The direction is robust across all three; the significance is not. Estimator choice matters even more than earlier drafts admitted, and we pre-specified none: median-of-ratios 3.40 $\times$, ratio-of-medians 4.31 $\times$, ratio-of-totals 2.64 $\times$; all reported. (The previous draft published 3.40 / 3.50 / 2.83 here; the 3.50 reproduces under no reconstruction we found, and the 2.83 matches the superseded \$45.76 imputation.)

Per-pair gap: mean \$25.15, median \$22.18. *No crossover exists in this data* — $N = \text{upfront/saving}$ is undefined when the saving is negative on every pair. Our design cannot test the *learning* half of the pre-registered hypothesis at all: cells are memoryless by construction.

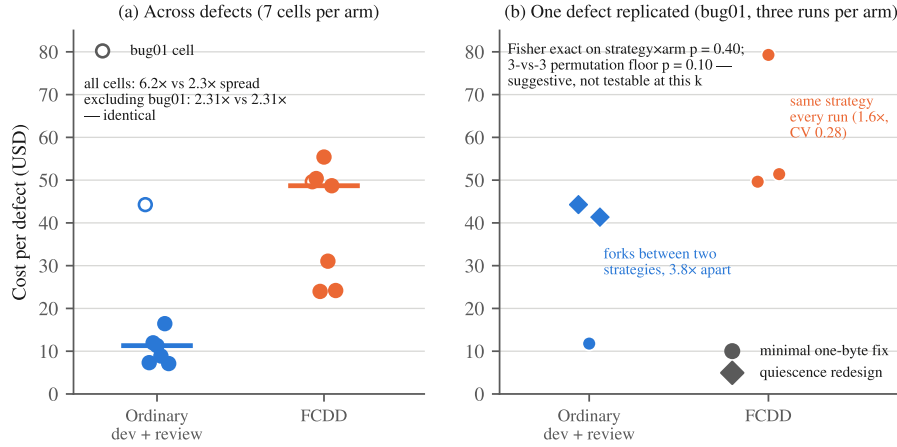


Fig. 4. (a) Per-defect costs by arm; the bug01 cell (hollow) carries the entire across-defect dispersion contrast. (b) The bug01 replication: three runs per arm under the final instrument; marker shape is the solution strategy the run converged on. Arm A forks between a minimal one-byte repair and an unrequested redesign 3.8 $\times$ apart; Arm B produced the minimal, byte-identical repair in all three runs.

6.2 Predictability (Goal 1a): dispersion, and what replication showed

This is the section the method exists to win, so we hold it to the same standard the cost section survived.

Across defects, the contrast rests on one cell. With all seven cells, ordinary development’s cost spread is 6.23 $\times$ (CV 0.86) against FCDD’s 2.31 $\times$ (CV 0.33) — the shape the thesis predicts. Excluding bug01, the spreads are *identical* at 2.31 $\times$ (CVs 0.34 and 0.36). A predictability claim whose entire evidentiary weight sits in one of seven cells is not a claim about the method; it is a claim about that cell (Fig. 4a).

Within the one replicated cell, the arms differ in kind, not just spread. Our pre-registered rule 5 required replicating bug01 (arm gap 10.8%); we ran three runs per arm under the final instrument (Fig. 4b). Arm A produced two different solutions: a *minimal one-byte repair* restoring the pristine binary (\$11.78), or an unrequested *redesign of the quiescence policy* (\$44.27, \$41.35) — the same redesign two graders scored at or below the correctness gate floor for an arithmetic underflow and a search-wide blast radius, and which the sealed acceptance script passed identically. Arm B produced the minimal, byte-identical repair in *all three runs* (\$49.63, \$79.28, \$51.40; spread 1.60 $\times$, CV 0.28). Arm A’s cost is not scattered by noise; it is determined by which strategy the run converges on (within-mode spreads 1.07 $\times$ and — including the instrument-v3 run, whose cost

is non-comparable but whose strategy datum is — $1.36\times$; between modes $3.8\times$). This is distinct from the $\sim 30\times$ stochastic token variance reported for agentic coding tasks [5]: it is bimodal strategy selection, interpretable rather than noise.

What this supports, stated exactly. The strategy table (Arm A: 1 minimal / 2 redesign; Arm B: 3 / 0) gives Fisher exact $p = 0.40$ (0.43 including the v3 strategy datum). At three runs per arm, *no dispersion statistic can reach significance* — the best attainable two-sided p in any 3-vs-3 permutation test is 0.10. And the oracle asymmetry (Sect. 4.2) offers a rival explanation requiring no method effect at all: an arm holding a pristine-derived executable model may converge on the pristine-restoring repair *because it holds the model*, not because a contract disciplines its strategy. The honest classification: *a directional observation, consistent with the method’s design intent, on one defect, confounded by the oracle, and untestable at this replication depth.* It is the right hypothesis for the next experiment (Sect. 9); it is not a finding of this one.

Solution-matched cost, reported but not as an estimate. Restricting to runs where both arms produced the byte-identical repair leaves three Arm B runs against one Arm A run (\$11.78); the three ratios are $4.21\times$, $4.36\times$, $6.73\times$, quoted individually because a denominator of one is not an estimator. The $1.12\times$ in Table 1 — the study’s narrowest ratio — compares FCDD’s minimal fix against Arm A’s expensive redesign; matched on solution it is closer to the study’s widest.

What the cost data adds to Goal 1. If “predictable” includes “you can budget for it”, the seven-pair table carries one more observation: Arm A’s per-defect cost has a long right tail (six cells in \$7–\$17, one at \$44), and the unbounded-review pathology shows what the tail looks like without a stopping rule — at least \$128.62 on a defect whose fix was one byte and landed in the first pass. FCDD’s band is narrower relative to its own scale but sits three to five times higher.

6.3 Quality and unfounded changes (Goal 1b)

The earlier comparison was withdrawn (order-confounded); we re-ran the grading with *each of the seven pairs graded twice, once in each presentation order, by fourteen independent graders*. Presentation preference came out exactly 7 first / 7 second, and all seven pairs returned the same verdict in both orders — evidence of grader reliability the single-rater design could not provide.

FCDD was favoured on 6 of 7 pairs ($p = 0.125$). The paired composite difference is $+0.17$ with a 95% CI of $[-0.44, +0.78]$ — the interval spans zero, and the study is not powered to make a quality claim (Fig. 5). Two further caveats make even the directional reading unsafe. First, the holistic preference item was never pre-registered — the protocol registers only the three 1–5 axes and the no-axis-below-3 gate — and it contradicts the graders’ own rubric totals on three of seven pairs, all three toward FCDD; scored on the pre-registered instrument the pairs split 3/2 with 2 ties, $p = 1.00$. Second, the aggregate is carried entirely by one pair: excluding bug01 the arms tie at exactly 4.50/4.50

Table 2. Counterbalanced grading, axis means (1–5).

Axis	All 7 pairs		Excl. bug01	
	Arm A	FCDD	Arm A	FCDD
Correctness risk	4.36	4.50	4.67	4.42
Clarity	4.64	4.29	4.75	4.17
Test quality	4.14	4.86	4.08	4.92
Composite	4.38	4.55	4.50	4.50

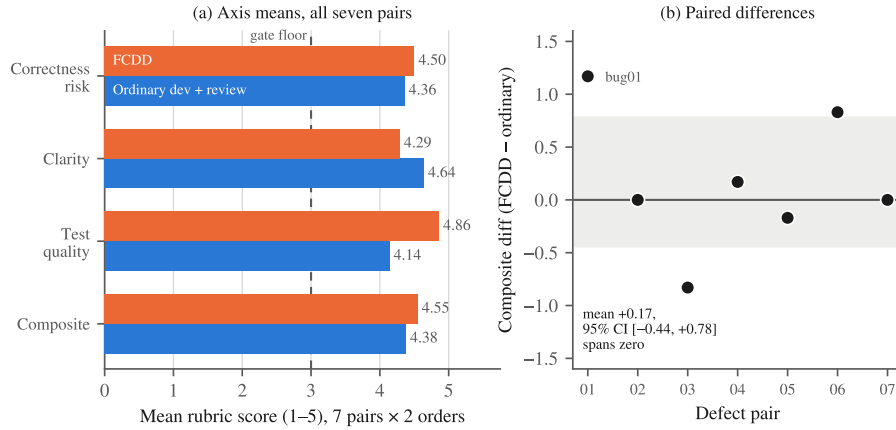


Fig. 5. (a) Axis means over the counterbalanced round (7 pairs $\times$ 2 orders). (b) Per-pair composite differences (FCDD – ordinary); the shaded band is the 95% CI, which spans zero. No grader assigned a composite; it is an unweighted mean of three ordinal axes, reported for description only.

and correctness risk reverses to 4.67 (ordinary) against 4.42 (FCDD) (Table 2). The only axis surviving exclusion is test quality — which mechanically rewards shipping more verification material (Sect. 5.5).

The advantage is concentrated in test quality and partly returned in clarity; graders repeatedly found FCDD’s evidence stronger but harder to read. On three of the seven defects the two source fixes were byte-for-byte identical, so the verdict turned entirely on verification. The defensible claim is narrow: FCDD did not produce better *fixes*; it produced better *evidence that the fix is right and will stay right* — seeding each fault as a permanent negative test, proving its detector fires by name, and in three cases closing a stale-evidence false-green in which the gate had been passing while the build’s own self-test failed.

What this measures against the hallucination-reduction goal. Goal 1 claims the contract suppresses unfounded changes. The graded record contains exactly one clear instance of the failure mode the goal targets: Arm A’s bug01 redesign — an unrequested rework of the quiescence policy that introduced an arithmetic-

underflow risk, scored 2 on correctness risk, *below the pre-registered gate floor of 3*, and passed the sealed acceptance script anyway. FCDD produced no graded submission below the floor, and on the replicated defect it never produced the redesign. That is the design intent made visible — once. Against it: both arms produced beyond-minimal changes on other defects (Arm A’s bug06 extra was a genuine latent-defect fix the grader valued; Arm B’s bug06 binary also differs from pristine), the quality interval spans zero, and the one clean instance sits on the same oracle-confounded cell as every other Goal-1 signal. *This experiment therefore does not establish hallucination reduction.* What it establishes is narrower and still worth having: the sealed symptom-level gate — the industry’s default arbiter — *cannot see the difference* between a careful repair and a risky redesign, because both pass it. A method that wants credit for suppressing unfounded changes must first deploy an instrument able to detect them; the contract’s clause-level diff is such an instrument, and the acceptance script is not.

6.4 Correctness and beyond-gate findings

Every defect was repaired correctly by both arms under the final instrument, confirmed by sealed acceptance scripts. This is weaker evidence than it sounds: the scripts were written by the agent that injected the faults, from the same understanding of the engine’s semantics, so a shared misconception is undetectable — the identical common-mode residual the paper quotes for FCDD and had not applied to its own instrument. Both arms produced findings beyond the reported defect: FCDD systematically (including a real defect in its own delivered package — a gate measured against a stale binary), and ordinary review found a latent transposition-table defect and an unadjudicated terminal-position path. “Finds more than it was asked to” is not FCDD-specific.

7 Discussion

7.1 Goal 1: predictability — designed for, not demonstrated

The method’s purpose receives, from this experiment, the verdict *consistent but unproven*. Every Goal-1 signal points the same way — the across-defect spread, the single-mode replication, the absence of any FCDD submission below the gate floor, the graders’ 6-of-7 preference — and not one survives its own caveat: the spread contrast lives in one cell; the replication depth caps attainable significance at $p = 0.10$; the gate-floor instance is a single event; the preference interval spans zero; and the oracle asymmetry predicts most of it with no method effect at all. Had we led with Goal 1 and reported these as findings, we would be doing with dispersions exactly what we criticise cost claims for doing with anecdotes. What the study does contribute to Goal 1 is sharper than a null: it identifies the phenomenon worth measuring (bimodal strategy selection, not continuous cost noise), the instrument capable of measuring it (solution-mode coding by binary

diff against pristine), the confound to remove (a cost-matched pristine-derived artefact for the control arm), and the replication depth required. Section 9 turns that into a design.

7.2 Goal 2: cost — demonstrated, with one imputation caveat

FCDD was more expensive on every one of seven matched faults, holding a free pristine-derived oracle its competitor lacked, and it never amortised. The direction survives every cost basis and sensitivity; the significance ($p = 0.016$ vs 0.125) rests on one transcript-recovery imputation we performed on our own study. The overhead on this workload was three to five times — not the order of magnitude folklore sometimes suggests, and not the near-zero that would make the method free to adopt. Since the contract never amortised, the trade the thesis proposes is real only if Goal 1’s product exists; this study priced the premium without confirming the insurance pays out.

7.3 The measurement lessons

The result most likely to transfer is neither ratio. This study produced six confident, plausible, wrong findings, and every one was caught by a mechanical check independent of the agent that produced it — three by our own gates and transcripts, two by adversarial review of the finished draft, and one (the Sect. 5.3 regression) by the re-derivation discipline this revision imposed. Several of the errors favoured the conclusion we were drifting toward. The concrete lessons for measuring agent-based development: reconcile self-reported cost against transcripts; report reviewer invocation counts; control the review termination rule (it moved one cell $17.6\times$); randomise presentation order in grading; and re-derive every published number from the deposited artefacts before each revision, because one of ours could not be.

7.4 An illustrative parallel: the same model, with and without a solver

A side experiment on unrelated infrastructure offers a parallel worth stating, with the caveat that it is an illustration and not evidence: it was not controlled, each cell ran once, and it measures chess rather than repair.

The same model played a five-game ladder against a deterministic 1–5 ply chess engine twice: once choosing every move from its own judgement, once with Stockfish 18 at fixed depth selecting the moves. Unaided it went 3–2, and unpredictably so—the two losses were not at the two hardest levels (it lost at levels 3 and 5, winning level 4 in 73 moves). Aided it went 5–0, with the won games roughly halved in length (27, 26, 15, 36 and 45 moves).

The interesting part is *where* the solver helped. The agents logged their own candidate move before each engine query. At level 2 the engine twice averted material loss; at levels 3, 4 and 5 it never rescued anything—the model’s own

worst candidate was still winning at every decision point. The solver removed a tail; it did not raise a mean.

That is the shape this study measured, and it suggests why. **A chess engine is a generator; a proof kernel is a filter.** Stockfish proposes a better move than the model would have chosen. Lean proposes nothing—it refuses claims that do not follow. FCDD therefore leaves the generator untouched and adds a filter, which predicts exactly what we observed: dispersion narrows (§6.2) while the paired quality difference stays indistinguishable from zero (§6.3). A filter removes the catastrophic tail; it does not make the typical repair better. Our null result on quality is not a failure to detect an effect that ought to be there—it is what a filter-shaped intervention should produce.

The parallel also marks its own limits, and they cut against the method rather than for it. Chess supplies a total, cheap, decidable oracle: Stockfish evaluates *any* legal position, and the objective—win—is given. Design supplies none of that. A contract constrains only what it states, the statement must be written by the same fallible agent whose work it is meant to bound, and writing it is where the cost and the errors concentrate. The seven cells here are the direct evidence: every one required *extending* the contract by 12 to 35 theorems before the fault was even expressible (§4.2), and an earlier draft’s claim that the contract covered all seven faults by construction was false and withdrawn.

The honest reading is narrow: where an external, machine-checked oracle covers the decision, it converts a variable outcome into a bounded one at a price. Chess is the easy case, because the oracle is total and free. Formal contracts are the hard case, because the oracle must first be built—and this paper’s headline cost is largely the bill for building it.

8 Threats to Validity

Sample size: seven defects, $k = 1$ except one cell at $k = 3$, one codebase, one language, one model. *Oracle asymmetry* (Sect. 4.2): FCDD received a pristine-derived executable model; Arm A received nothing comparable. It did not cover the faults by construction (every cell extended it), but it is the most consequential design flaw and supplies a method-free explanation for every Goal-1 signal. *Instrument instability*: the prompt changed four times; only final-version cells are compared. Version purity was achieved; reproducibility was not demonstrated — the one replicated cell spans $3.8\times$ across runs within the same arm and instrument. *Interruption asymmetry*: from the deposited attempt records, 6 of 7 Arm B and 2 of 7 Arm A cells required a resume (previously published as “every/none”, then “5 of 7”; only this value derives from the ledger). The completing-attempt proxy gives $1.14\times$, $1.54\times$, $3.95\times$, $4.04\times$ (median $2.75\times$); it is a floor, not a bracket. One Arm B bug01 replicate (\$79.28) carries resumption overhead inside the Goal-1 dispersion data. *Review-budget asymmetry*: the bounded rule caps rounds, not invocations, granting Arm B $1.8\times$ the review effort. *Gate common-mode*: acceptance scripts were written by the fault-injecting agent — and Sect. 6.3 shows the gate passing a below-floor redesign, so the

weakness is demonstrated, not conjectured. *Grading*: label-blind, not treatment-blind (37:0 signature terms). Counterbalancing removed order confounding, but each pair $\times$ order was judged once: order-consistency, not inter-rater reliability. Graders ran on a different model from the arms and their reviewers — an earlier draft asserted a shared model family, which is wrong for this data. *Population*: single-byte seeded faults are individually localisable, plausibly favouring the cheaper method; none is a fault of omission, the class that lies outside any contract. *Target-language externality*: every cell repaired one language, and an unusual one: we assume hand-written Z80 assembly is far sparser in LLM training corpora than mainstream languages — a plausible but *unverified* premise: the training composition of the models used here is not public, and neither work cited below measures Z80’s share. What the literature does establish is the axis: multi-language benchmarks were built expressly to expose the effect of language frequency [10], and rebalancing the training distribution helps low-resource languages, though the absolute effect is modest — the headline 66.48% is a *relative* pass@ k improvement; the underlying absolute pass@100 moves (4B model, natural vs. balanced) run from -3.1 points (C#, which regresses) to $+13.0$ (Julia), mean $\approx +6$ [22] — so the frequency axis is real but neither large in absolute terms nor monotone. Two effects follow, with stateable expected signs. (i) *Sparcity widens the unguided arm*: with fewer memorised idioms to recall, more of an unconstrained session is search rather than recall, so part of Arm A’s dispersion — and therefore of the Goal-1 contrast — may be a property of Z80, not of the missing contract (the sign holds for within-strategy wander; for strategy *forking* it is genuinely ambiguous, since a rich language also offers more memorised redesigns to fork to). (ii) *Closed semantics flatter the contract*: Z80’s fixed instruction set, absence of runtime and library surface, and small machine state make a finite-scope contract unusually cheap to build (\$21.75) and unusually binding; a large-runtime language would cost more to specify and bind less. Jointly the expected direction differs by goal: a mainstream-language replication should *raise* the cost ratio (recall cheapens Arm A while specification surface raises Arm B’s contract side — our measured 3–5 $\times$ is a plausible floor, consistent with Sect. 2’s cheapest-tier argument) and *shrink* the dispersion contrast (a narrower unguided arm against a less binding contract) — on language grounds the cost finding is conservative and the predictability observation anti-conservative. Effect (ii) also compounds the oracle asymmetry (Sect. 4.2) rather than sitting orthogonal to it: a pristine-derived oracle is worth most exactly where the model’s prior knowledge is thinnest. None of this is testable here — language never varies across any cell; the nearest within-study proxy, fault subsystem, is not language variation at all (for the record, Arm A’s spread is 1.68 $\times$ within the four rule-layer cells against 3.92 $\times$ within the three search/eval cells — carried by bug01 again and confounded with defect difficulty and contract coverage). The natural language of prompts and reasoning — the secondary reading of “output language” — was English throughout and identical across arms, so it cannot contribute to a between-arm contrast. Item (2) of Sect. 9 specifies the replication that would make the language axis measurable. *Contamination*: the orchestrator

read session self-reports while ingesting results and unsealed the key at grading. *Conflict of interest:* see the declarations; an external audit found eight discretionary choices, seven favouring the authors’ own method.

9 Future Work: Buying More Predictability

The question this study leaves is what would make Goal 1 decidable, and what each candidate improvement would buy per dollar.

(1) *Power the predictability claim before making it.* The phenomenon is identified: bimodal strategy selection, detectable mechanically (binary diff against pristine classifies minimal-fix vs redesign at zero marginal cost). A decidable design needs $k \geq 6$ runs per arm per defect on at least three defects (at $k = 3$ the attainable two-sided p floors at 0.10; at $k = 6$ a clean 6/0-vs-3/3 strategy split reaches $p \approx 0.01$); a pre-registered dispersion estimator (log-cost CV with a permutation test); and the control arm equipped with a cost-matched pristine-derived artefact ($\approx \$21.75$ of characterisation tests) so the oracle confound is spent symmetrically. At observed per-cell prices this is roughly \$1,400–\$2,000 of runs — comparable to this entire study, for one decidable bit. That is the actual price of the predictability claim.

(2) *A cross-language replication prices the language confound.* The target-language threat (Sect. 8) is untestable in this design because the language being repaired never varies, and it bears on the two goals with opposite signs — it argues the cost ratio is a floor and the dispersion contrast is inflated. The decisive experiment manipulates language and holds everything else as fixed as a different codebase permits: seed the same seven fault classes — all language-portable (an inverted guard, an unreset counter, a generation mask, an evaluation sign, a rights-bookkeeping slip, a cache depth bound, a terminal misclassification) — into a chess engine of comparable scope in at least one high-resource language (Python or C; compact open-source engines exist in both), build the contract fresh for that engine (the build cost is itself a primary measurement: effect *ii*) predicts it rises with runtime and library surface), and run both arms under item (1)’s controls. At the measured per-cell means (\$15.33 Arm A, \$40.47 Arm B) one added language costs $\approx \$390$ of runs at $k = 1$ and $\approx \$1,170$ at $k = 3$, plus contract build and seeding — the same order as item (1), with which it shares runs if the $k \geq 6$ defects are drawn from the new language. It can settle whether the unguided arm’s spread and strategy-forking rate move with training-data share (effect *i*) and whether contract build and per-defect extension costs track semantic surface (effect *ii*) — each a signed prediction stated before any data exist. It cannot separate language from codebase beyond the fault-class matching, and it says nothing about faults of omission or other model families. A third, mid-frequency language turns the two-point contrast into a trend for roughly another \$1,200.

(3) *Certified extraction: from sampling agreement to correctness by construction.* Case 01’s deepest structural weakness is that spec, twin and bridge *sample* each other’s agreement. The Rocq/Coq route — specify the contract in Rocq, prove the properties deductively, and extract executable OCaml from the proved specification, as CompCert does at scale [19] — replaces the twin-and-bridge beats with an artefact correct by construction relative to its spec. For Goal 1 the buy is qualitative: for covered properties the failure mode shifts from *detected downstream* to *unrepresentable*, and the per-defect verification cost for that class approaches zero — precisely the regime in which this study’s never-amortising upfront cost could amortise; the crossover arithmetic that Sect. 6.1 found undefined becomes well-defined the moment one defect class’s marginal cost drops below the ordinary arm’s. The cost: deductive proof effort well above the \$21.75 finite-scope tier we priced (a floor, per Sect. 2); and extraction certifies the extracted core against *its own spec* only — it does not certify the spec (bug04’s vacuous antisymmetry clause would survive extraction untouched), the FFI boundary, the numeric-width remapping, or the parts left in Z80 and JavaScript. Those leaks are exactly where “extracted therefore correct” fails in practice and must be budgeted as first-class work.

(4) *An independent oracle attacks the common-mode residual.* Both the method’s own disclaimer and our gate finding reduce to one sentence: *spec and checker share an author*. The cheapest structural attack is a second, independently-authored specification to differential-test against. One partially exists in the *Chess.lean* project¹. Its rules/legality/mate layer overlaps our contract’s C1–C6/C11; the strategy layer our expensive cells needed (search termination, eval anchoring, TT soundness) is out of its scope. Highest-value use is therefore as an *independent oracle*: differential-test our spec’s move generation and terminal decisions against it — days of encoding reconciliation for the first evidence about spec correctness that does not pass through this study’s single author.

(5) *The Lean-to-Rocq import path is a spike, not a plan.* The tempting composite — reuse *Chess.lean* and get certified extraction via *rocq-lean-import*² — is the path the tool’s own documentation says is most likely to fail. The correct next step is a bounded spike (≤ 1 session): does *Chess.lean*’s dependency set survive `lean4export` plus the importer, and does Rocq extraction produce OCaml for the chess definitions specifically? Any red answer falls back to (3)’s Rocq-native spec with *Chess.lean* kept in its oracle role.

¹ <https://github.com/dwrensha/Chess.lean> — a Lean 4 formalisation, self-described as incomplete: board representation, move legality, turn tracking, checkmate detection, oriented toward proving mate puzzles. Verified 2026-07-30; re-verified 2026-08-06.

² <https://github.com/rocq-community/rocq-lean-import> — translates Lean export files into Rocq declarations. Its own README (verified 2026-08-06) labels it an experimental alpha, states the translation breaks extraction for some constructs, requires conversion checking disabled for the Lean stdlib, and reports mathlib intractable (11,867 of 66,400 entries skipped even with errors tolerated).

(6) *Cheap contract-side hardening from this study’s own failures. Minimal-repair obligation:* make “the smallest change satisfying the violated clause” an explicit, diff-size-budgeted contract obligation — targets the bug01 redesign fork directly at the cost of a prompt line and a gate check; its risk is forbidding the beneficial latent-defect fix Arm A’s bug06 run produced, so the organic-bonus lane must remain a sanctioned exit. *Specification mutation testing:* bug04 proved a clause can be vacuous against an entire fault class; systematically mutating the twin and requiring each mutation to trip a clause would have found that vacuity at build time. *Invocation-capped review:* capping invocations rather than rounds makes the review budget an actual budget (Sect. 5.3).

(7) *Instrument the fix/verification split.* This study cannot say what fraction of FCDD’s 3–5× premium buys verification rather than repair, because cells report only totals. Timestamping the moment the fix first passes the acceptance script would split every cell into time-to-fix and verification tail — the number a practitioner budgeting FCDD adoption actually needs, and the number that decides whether future effort should go to cheaper proving or faster localisation.

10 Conclusions

(1) Both methods reliably repaired real assembly defects; every cell under the final instrument produced an acceptance-passing fix. (2) *Goal 2 (cost) is answered.* FCDD was more expensive on 7 of 7 matched pairs ($p = 0.016$, contingent on one disclosed imputation; direction robust regardless), median ratio 4.74× on raw tokens and ≈ 3.3 –3.4× on priced bases, interval [1.12, 6.88], and its upfront contract never amortised. (3) *Goal 1 (predictability) is not answered, and we decline to pretend otherwise.* The across-defect dispersion contrast rests on a single cell; the replicated cell shows the predicted shape — FCDD single-mode, the ordinary arm forking between a minimal fix and a below-gate-floor redesign — at a depth where no test can reach significance (Fisher $p = 0.40$; attainable floor 0.10) and under an oracle asymmetry that predicts the same shape without any method effect. It is a hypothesis with a mechanism and an instrument, not a finding. (4) *No quality claim is made.* FCDD was favoured on 6 of 7 pairs ($p = 0.125$) and the paired composite CI [−0.44, +0.78] spans zero; excluding one pair the arms tie exactly, and the grading was label-blind but not treatment-blind. What survives is narrow: identical or equivalent fixes, better evidence — and one demonstration that the sealed gate cannot distinguish a careful repair from a risky redesign, which is the strongest argument in this data for contract-shaped instruments even if not for the contract’s price. (5) *Measurement discipline dominates.* Report reviewer counts, reconcile costs against transcripts, control the review termination rule, randomise grading order, and re-derive every number before every draft — we failed each of these at least once, and each failure produced a publishable-looking result that was false.

Data availability. Deposited: the protocol with all amendments (A1–A10), the per-cell ledger, the raw session records, the prompt packs, the grading packets

and verdicts, the contract, and the analysis and figure scripts. Not deposited: the sealed answer key, the acceptance scripts, the seeded variants, the subject engine and the emulator harness (a SHA-256 manifest of 206 files is deposited). The per-cell workspaces on which reviewer counts, binary comparisons, gate verdicts and per-cell theorem counts depend sit outside the repository, so those specific claims are not independently checkable from the artefact.

AI usage. This study measures AI agents and was conducted by them: all sessions, both arms, all reviewers and all graders were LLM agents; orchestration, analysis and this manuscript were produced by an LLM agent under human direction. The human operator set the research question, adjudicated amendments and directed each phase.

References

1. Aleithan, R., Xue, H., Mohajer, M.M., Nnorom, E., Uddin, G., Wang, S.: SWE-Bench+: Enhanced coding benchmark for LLMs. arXiv preprint arXiv:2410.06992 (2024)
2. Amusuo, P.C., Cochell, O., Le Lievre, T., Patil, P.V., Machiry, A., Davis, J.C.: Do unit proofs work? an empirical study of compositional bounded model checking for memory safety verification. arXiv preprint arXiv:2503.13762 (2025). <https://doi.org/10.48550/arXiv.2503.13762>
3. Andrews, J.H., Briand, L.C., Labiche, Y.: Is mutation an appropriate tool for testing experiments? In: Proc. 27th Int. Conf. on Software Engineering (ICSE). pp. 402–411 (2005). <https://doi.org/10.1145/1062455.1062530>
4. Bacchelli, A., Bird, C.: Expectations, outcomes, and challenges of modern code review. In: Proc. 35th Int. Conf. on Software Engineering (ICSE). pp. 712–721 (2013). <https://doi.org/10.1109/ICSE.2013.6606617>
5. Bai, L., Huang, Z., Wang, X., Sun, J., Mihalcea, R., Brynjolfsson, E., Pentland, A., Pei, J.: How do AI agents spend your money? analyzing and predicting token consumption in agentic coding tasks. arXiv preprint arXiv:2604.22750 (2026)
6. ter Beek, M.H., Ferrari, A.: Empirical formal methods: Guidelines for performing empirical studies on formal methods. *Software* **1**(4), 381–416 (2022). <https://doi.org/10.3390/software1040017>
7. Beller, M., Bacchelli, A., Zaidman, A., Juergens, E.: Modern code reviews in open-source projects: Which problems do they fix? In: Proc. 11th Working Conf. on Mining Software Repositories (MSR). pp. 202–211 (2014). <https://doi.org/10.1145/2597073.2597082>
8. Berry, D.M., Tichy, W.F.: Comments on “formal methods application: An empirical tale of software development”. *IEEE Transactions on Software Engineering* **29**(6), 567–571 (2003). <https://doi.org/10.1109/TSE.2003.1205183>
9. Bornholt, J., Joshi, R., Astrauskas, V., Cully, B., Kragl, B., Markle, S., Sauri, K., Schleit, D., Slatton, G., Tasiran, S., Van Geffen, J., Warfield, A.: Using lightweight formal methods to validate a key-value storage node in Amazon S3. In: Proc. 28th ACM Symp. on Operating Systems Principles (SOSP). pp. 836–850 (2021). <https://doi.org/10.1145/3477132.3483540>
10. Cassano, F., Gouwar, J., Nguyen, D., Nguyen, S., Phipps-Costin, L., Pinckney, D., Yee, M.H., Zi, Y., Anderson, C.J., Feldman, M.Q., Guha, A., Greenberg, M.,

- Jangda, A.: MultiPL-E: A scalable and extensible approach to benchmarking neural code generation. arXiv preprint arXiv:2208.08227 (2022)
11. Haeri, A., Ghelichi, M.: Specification grounding drives test effectiveness for LLM code. arXiv preprint arXiv:2607.06636 (2026)
 12. Hannay, J.E., Dybå, T., Arisholm, E., Sjøberg, D.I.K.: The effectiveness of pair programming: A meta-analysis. *Information and Software Technology* **51**(7), 1110–1122 (2009). <https://doi.org/10.1016/j.infsof.2009.02.001>
 13. Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K.: SWE-bench: Can language models resolve real-world GitHub issues? In: Proc. 12th Int. Conf. on Learning Representations (ICLR) (2024)
 14. Just, R., Jalali, D., Inozemtseva, L., Ernst, M.D., Holmes, R., Fraser, G.: Are mutants a valid substitute for real faults in software testing? In: Proc. 22nd ACM SIGSOFT Int. Symp. on Foundations of Software Engineering (FSE). pp. 654–665 (2014). <https://doi.org/10.1145/2635868.2635929>
 15. Kapoor, S., Stroebl, B., Siegel, Z.S., Nadgir, N., Narayanan, A.: AI agents that matter. arXiv preprint arXiv:2407.01502 (2024)
 16. Karampatsis, R.M., Sutton, C.: How often do single-statement bugs occur? the ManySStuBs4J dataset. In: Proc. 17th Int. Conf. on Mining Software Repositories (MSR) (2020). <https://doi.org/10.1145/3379597.3387491>
 17. Klein, G., Elphinstone, K., Heiser, G., Andronick, J., Cock, D., Derrin, P., Elkaduwe, D., Engelhardt, K., Kolanski, R., Norrish, M., Sewell, T., Tuch, H., Winwood, S.: seL4: Formal verification of an OS kernel. In: Proc. 22nd ACM SIGOPS Symp. on Operating Systems Principles (SOSP). pp. 207–220 (2009). <https://doi.org/10.1145/1629575.1629596>
 18. Ko, A.J., LaToza, T.D., Burnett, M.M.: A practical guide to controlled experiments of software engineering tools with human participants. *Empirical Software Engineering* **20**(1), 110–141 (2015). <https://doi.org/10.1007/s10664-013-9279-3>
 19. Leroy, X.: Formal verification of a realistic compiler. *Communications of the ACM* **52**(7), 107–115 (2009). <https://doi.org/10.1145/1538788.1538814>
 20. Mäntylä, M.V., Lassenius, C.: What types of defects are really discovered in code reviews? *IEEE Transactions on Software Engineering* **35**(3), 430–448 (2009). <https://doi.org/10.1109/TSE.2008.71>
 21. Newcombe, C., Rath, T., Zhang, F., Munteanu, B., Brooker, M., Deardouff, M.: How Amazon Web Services uses formal methods. *Communications of the ACM* **58**(4), 66–73 (2015). <https://doi.org/10.1145/2699417>
 22. Orlanski, G., Xiao, K., Garcia, X., Hui, J., Howland, J., Malmaud, J., Austin, J., Singh, R., Catasta, M.: Measuring the impact of programming language distribution. In: Proc. 40th Int. Conf. on Machine Learning (ICML) (2023)
 23. Papadakis, M., Henard, C., Harman, M., Jia, Y., Le Traon, Y.: Threats to the validity of mutation-based test assessment. In: Proc. Int. Symp. on Software Testing and Analysis (ISSTA). pp. 354–365 (2016). <https://doi.org/10.1145/2931037.2931040>
 24. Papadakis, M., Shin, D., Yoo, S., Bae, D.H.: Are mutation scores correlated with real fault detection? In: Proc. 40th Int. Conf. on Software Engineering (ICSE). pp. 537–548 (2018). <https://doi.org/10.1145/3180155.3180183>
 25. Pfleeger, S.L., Hatton, L.: Investigating the influence of formal methods. *IEEE Computer* **30**(2), 33–43 (1997). <https://doi.org/10.1109/2.566148>
 26. Porter, A., Siy, H., Mockus, A., Votta, L.: Understanding the sources of variation in software inspections. *ACM Transactions on Software Engineering and Methodology* **7**(1), 41–79 (1998). <https://doi.org/10.1145/268411.268421>

27. Rigby, P.C., Bird, C.: Convergent contemporary software peer review practices. In: Proc. 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE). pp. 202–212 (2013). <https://doi.org/10.1145/2491411.2491444>
28. Rondon, P., Wei, R., Cambronero, J., Cito, J., Sun, A., Sanyam, S., Tufano, M., Chandra, S.: Evaluating agent-based program repair at Google. arXiv preprint arXiv:2501.07531 (2025)
29. Sadowski, C., Söderberg, E., Church, L., Sipko, M., Bacchelli, A.: Modern code review: A case study at Google. In: Proc. 40th Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP) (2018). <https://doi.org/10.1145/3183519.3183525>
30. Shi, L., Ma, C., Liang, W., Diao, X., Ma, W., Vosoughi, S.: Judging the judges: A systematic study of position bias in LLM-as-a-judge. In: Proc. AACL-IJCNLP (2025), arXiv:2406.07791 (2024)
31. Sobel, A.E.K., Clarkson, M.R.: Formal methods application: An empirical tale of software development. *IEEE Transactions on Software Engineering* **28**(3), 308–320 (2002). <https://doi.org/10.1109/32.991322>
32. Sobel, A.E.K., Clarkson, M.R.: Response to “comments on ‘formal methods application: An empirical tale of software development’”. *IEEE Transactions on Software Engineering* **29**(6), 572–575 (2003). <https://doi.org/10.1109/TSE.2003.1205184>
33. Staples, M., Jeffery, R., Andronick, J., Murray, T., Klein, G., Kolanski, R.: Productivity for proof engineering. In: Proc. 8th ACM/IEEE Int. Symp. on Empirical Software Engineering and Measurement (ESEM) (2014). <https://doi.org/10.1145/2652524.2652551>
34. Wang, P., Li, L., Chen, L., Cai, Z., Zhu, D., Lin, B., Cao, Y., Kong, L., Liu, Q., Liu, T., Sui, Z.: Large language models are not fair evaluators. In: Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL). pp. 9440–9450 (2024). <https://doi.org/10.18653/v1/2024.acl-long.511>
35. Wang, R., Guo, J., Gao, C., Fan, G., Chong, C.Y., Xia, X.: Can LLMs replace human evaluators? an empirical study of LLM-as-a-judge in software engineering. In: Proc. Int. Symp. on Software Testing and Analysis (ISSTA) (2025). <https://doi.org/10.1145/3728963>
36. Woodcock, J., Larsen, P.G., Bicarregui, J., Fitzgerald, J.: Formal methods: Practice and experience. *ACM Computing Surveys* **41**(4), 1–36 (2009). <https://doi.org/10.1145/1592434.1592436>
37. Xia, C.S., Deng, Y., Dunn, S., Zhang, L.: Demystifying LLM-based software engineering agents. In: Proc. ACM on Software Engineering (FSE). vol. 2, pp. 801–824 (2025). <https://doi.org/10.1145/3715754>
38. Xia, C.S., Wei, Y., Zhang, L.: Automated program repair in the era of large pre-trained language models. In: Proc. 45th Int. Conf. on Software Engineering (ICSE). pp. 1482–1494 (2023). <https://doi.org/10.1109/ICSE48619.2023.00129>
39. Yang, J., Jimenez, C.E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., Press, O.: SWE-agent: Agent-computer interfaces enable automated software engineering. arXiv preprint arXiv:2405.15793 (2024)
40. Zheng, L., Chiang, W.L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E.P., Zhang, H., Gonzalez, J.E., Stoica, I.: Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In: Advances in Neural Information Processing Systems 36 (NeurIPS Datasets and Benchmarks) (2023)